

Serverless Notebook Spend Alert

Detector notebook — emails any user whose serverless notebook spend exceeds \$1,000 over the last 7 days.
Auth via Microsoft Graph + Managed Identity (no passwords, no secrets).

Circle K · Platform

v2.0 · May 2026

One-time setup — grant Mail.Send to the workspace identity

The notebook authenticates to Microsoft Graph using the Databricks workspace's managed identity. Your Azure AD admin grants that identity `Mail.Send` (Application permission) one time, with admin consent. After that, the notebook calls Graph with a token it gets at runtime — nothing stored in code or secrets.

Azure portal path: Azure AD → Enterprise Applications → find your Databricks workspace's managed identity → *Permissions* → *Add a permission* → Microsoft Graph → Application permissions → `Mail.Send` → *Grant admin consent for {tenant}*.

PowerShell equivalent:

```
Connect-MgGraph -Scopes "AppRoleAssignment.ReadWrite.All"

$msi = Get-MgServicePrincipal -Filter "displayName eq 'ck-databricks-prod-msi'"
$graph = Get-MgServicePrincipal -Filter "appId eq '00000003-0000-0000-c000-000000000000'"
$role = $graph.AppRoles | Where-Object Value -EQ "Mail.Send"

New-MgServicePrincipalAppRoleAssignment `
  -ServicePrincipalId $msi.Id `
  -PrincipalId        $msi.Id `
  -ResourceId         $graph.Id `
  -AppRoleId          $role.Id
```

MAIL.SEND IS TENANT-WIDE

The Application permission lets the identity send mail *as any user in the tenant*. To scope it down to a single mailbox (e.g. only `platform-cost@circlek.com`), have your Exchange admin add an *ApplicationAccessPolicy* restricting the identity to that mailbox. Standard practice for service identities.

The notebook

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Serverless Notebook Spend Detector – $1,000 / 7-day threshold
# MAGIC
# MAGIC Auth: Microsoft Graph + Managed Identity. No secrets required.

# COMMAND -----

# MAGIC %pip install --quiet azure-identity
# MAGIC %restart_python

# COMMAND -----

# MAGIC %md
# MAGIC ## 1. Configuration

# COMMAND -----

CONFIG = {
    "threshold_dollars":    1000,
    "lookback_days":       7,
    "workspace_host":      "https://<your-ws>.azuredatabricks.net",
    "sender_address":       "platform-cost@circlek.com",
    "platform_dl":         "platform-cost@circlek.com",
    "teams_channel_webhook": "",          # optional; leave empty to skip Teams rollup
    "exclude_users":       {"loadtest-sp@circlek.com",
                           "etl-runner-sp@circlek.com"},
    "dry_run":             True,          # flip to False after verifying output
    "audit_table":         "ops.cost_alerts.notification_log",
    "min_resend_hours":    24,           # do not re-email same user within window
}

# COMMAND -----

# MAGIC %md
# MAGIC ## 2. Detect – serverless notebook spend by user, last 7 days

# COMMAND -----

offenders_df = spark.sql(f"""
WITH window AS (
    SELECT
        u.identity_metadata.run_as                AS user_email,
        u.workspace_id,
        SUM(u.usage_quantity * lp.pricing.default) AS list_cost,
        SUM(u.usage_quantity)                     AS dbus,
        COUNT(DISTINCT u.usage_metadata.notebook_id) AS notebooks_used,
        COLLECT_SET(u.usage_metadata.notebook_path) AS notebook_paths
    FROM system.billing.usage u
    INNER JOIN system.billing.list_prices lp
```

```

    ON u.cloud                = lp.cloud
    AND u.sku_name            = lp.sku_name
    AND u.usage_start_time    >= lp.price_start_time
    AND (u.usage_end_time    <= lp.price_end_time OR lp.price_end_time IS NULL)
WHERE u.usage_date >= CURRENT_DATE() - INTERVAL {CONFIG['lookback_days']} DAYS
    AND u.sku_name LIKE '%SERVERLESS%'
    AND u.billing_origin_product = 'INTERACTIVE'
    AND u.usage_metadata.notebook_id IS NOT NULL
GROUP BY 1, 2
)
SELECT *
FROM window
WHERE list_cost > {CONFIG['threshold_dollars']}
ORDER BY list_cost DESC
""")

```

```

offenders = [r.asDict() for r in offenders_df.collect()
              if r["user_email"]
              and r["user_email"] not in CONFIG["exclude_users"]]

```

```

print(f"Found {len(offenders)} user(s) above ${CONFIG['threshold_dollars']} "
      f"over the last {CONFIG['lookback_days']} days.")

```

```

# COMMAND -----

```

```

# MAGIC %md

```

```

# MAGIC ## 3. Audit table + dedup (do not re-email within 24h)

```

```

# COMMAND -----

```

```

spark.sql(f"""
CREATE TABLE IF NOT EXISTS {CONFIG['audit_table']} (
    alert_time    TIMESTAMP,
    user_email    STRING,
    workspace_id  STRING,
    list_cost     DOUBLE,
    lookback_days INT,
    threshold     DOUBLE,
    channel       STRING,
    status        STRING
) USING DELTA
""")

```

```

recently_alerted = {
    r["user_email"]
    for r in spark.sql(f"""
        SELECT DISTINCT user_email
        FROM {CONFIG['audit_table']}
        WHERE channel = 'email'
            AND status = 'sent'
            AND alert_time >= CURRENT_TIMESTAMP()
              - INTERVAL {CONFIG['min_resend_hours']} HOURS
        """).collect()
}

```

```

}

# COMMAND -----

# MAGIC %md
# MAGIC ## 4. Send personalized email via Microsoft Graph

# COMMAND -----

import requests
from datetime import datetime
from azure.identity import DefaultAzureCredential

# DefaultAzureCredential walks the auth chain: managed identity first, then
# environment vars, then federated credentials. No secret in code.
credential = DefaultAzureCredential()
graph_token = credential.get_token("https://graph.microsoft.com/.default").token

GRAPH_SEND = (f"https://graph.microsoft.com/v1.0/users/"
              f"{CONFIG['sender_address']}/sendMail")

def send_email(to_addr, subject, html_body):
    if CONFIG["dry_run"]:
        print(f"[DRY-RUN] would email {to_addr}: {subject}")
        return "dry_run"
    response = requests.post(
        GRAPH_SEND,
        headers={
            "Authorization": f"Bearer {graph_token}",
            "Content-Type": "application/json",
        },
        json={
            "message": {
                "subject": subject,
                "body": {"contentType": "HTML", "content": html_body},
                "toRecipients": [{"emailAddress": {"address": to_addr}}],
            },
            "saveToSentItems": "false",
        },
        timeout=30,
    )
    response.raise_for_status()
    return "sent"

def email_body(row):
    paths = "<br>".join(
        f"&mdot; <code>{p}</code>"
        for p in (row["notebook_paths"] or [])[:10]
    )
    return f"""
<h2>Heads up – your serverless notebook spend</h2>
<p>Your serverless interactive notebook usage on workspace
    <code>{row['workspace_id']}</code> totaled

```

```

        <b>${row['list_cost']:.2f}</b> ({row['dbus']:.1f} DBUs)
        over the last {CONFIG['lookback_days']} days, across
        {row['notebooks_used']} notebook(s).</p>
    <p>This crossed the ${CONFIG['threshold_dollars']} threshold the
        platform team monitors. No action is required if this was expected –
        this is a personal heads-up so you have the same view as the
        platform team.</p>
    <p><b>Notebooks involved:</b><br>{paths or "(no path captured)"}</p>
    <p>For a per-notebook breakdown, ask <b>Genie Code in agent mode</b>:
        <em>"Which notebooks did I run in the last 7 days that incurred
        those costs?"</em></p>
    <p><a href="{CONFIG['workspace_host']}/sql/history">Open Query History</a></p>
    <p style="color:#888;font-size:11px;">
        Auto-generated by the CK platform team · Reply to
        {CONFIG['platform_dl']} if this fired in error.
    </p>
    """"

```

```

audit_rows = []
sent_count, skipped_count = 0, 0

for row in offenders:
    user = row["user_email"]
    if user in recently_alerted:
        skipped_count += 1
        audit_rows.append((datetime.utcnow(), user, row["workspace_id"],
                           float(row["list_cost"]),
                           CONFIG["lookback_days"],
                           float(CONFIG["threshold_dollars"]),
                           "email", "skipped_recent"))
        continue
    subject = (f"[CK Serverless] Your notebook spend crossed "
               f"${CONFIG['threshold_dollars']} ({CONFIG['lookback_days']}d)")
    status = send_email(user, subject, email_body(row))
    sent_count += 1
    audit_rows.append((datetime.utcnow(), user, row["workspace_id"],
                       float(row["list_cost"]),
                       CONFIG["lookback_days"],
                       float(CONFIG["threshold_dollars"]),
                       "email", status))

if audit_rows:
    audit_df = spark.createDataFrame(
        audit_rows,
        schema="alert_time TIMESTAMP, user_email STRING, workspace_id STRING, "
               "list_cost DOUBLE, lookback_days INT, threshold DOUBLE, "
               "channel STRING, status STRING"
    )
    audit_df.write.mode("append").saveAsTable(CONFIG["audit_table"])

print(f"sent: {sent_count}, skipped_recent: {skipped_count}, "
      f"total_offenders: {len(offenders)}")

```

```

# COMMAND -----

# MAGIC %md
# MAGIC ## 5. Teams channel rollup (optional, skipped if webhook URL is empty)

# COMMAND -----

if CONFIG["teams_channel_webhook"] and offenders and not CONFIG["dry_run"]:
    summary = "\n".join(
        f"- **{r['user_email']}** · ${r['list_cost']:.2f} · "
        f"{r['notebooks_used']} notebook(s)"
        for r in offenders[:20]
    )
    extra = (f"\n\n...and {len(offenders) - 20} more_"
            if len(offenders) > 20 else "")
    card = {
        "type": "message",
        "attachments": [{
            "contentType": "application/vnd.microsoft.card.adaptive",
            "content": {
                "type": "AdaptiveCard", "version": "1.4",
                "body": [
                    {"type": "TextBlock", "size": "Medium", "weight": "Bolder",
                     "text": (f"Serverless notebook spend > "
                             f"${CONFIG['threshold_dollars']} "
                             f"({CONFIG['lookback_days']}d) - "
                             f"{len(offenders)} user(s)")),
                    {"type": "TextBlock", "wrap": True, "text": summary + extra}
                ]
            }
        }]
    }
    requests.post(CONFIG["teams_channel_webhook"], json=card, timeout=10)
    print("Teams rollup posted.")

# COMMAND -----

dbutils.notebook.exit(
    f"OK - {sent_count} emailed, {skipped_count} skipped, "
    f"{len(offenders)} total offenders"
)

```

Deployment

- 1 Confirm the one-time admin grant above is done. Verify the workspace's managed identity has Mail.Send on Microsoft Graph in Azure AD.
- 2 Save the notebook to /Shared/cost-alerts/spend-detector-1000-7d . Confirm the running identity has CREATE TABLE on the ops.cost_alerts schema.
- 3 Run interactively with dry_run=True . Verify the offender count and check the [DRY-RUN] log lines name the right people.

- 4 Flip `dry_run=False` and run once manually. Confirm an email lands in a test mailbox and a row appears in `ops.cost_alerts.notification_log`.
- 5 Create a Lakeflow Job: notebook task, serverless compute, cron `0 0 7 ? * * *` (07:00 daily). On failure → email `platform-cost@circlek.com`.

IF DEFAULTAZURECREDENTIAL CAN'T FIND AN IDENTITY ON SERVERLESS

On classic compute the Access Connector's managed identity is reachable via IMDS automatically. Serverless compute is more locked down — if the chain fails, the fallback is to register an Azure AD app, grant it `Mail.Send` (same as above), set up Workload Identity Federation between Databricks and that app (zero secrets), and set `AZURE_CLIENT_ID` / `AZURE_TENANT_ID` as job environment variables. The notebook code stays identical.

WHY DEDUP MATTERS

The 24-hour `min_resend_hours` window prevents the same user from being re-emailed every day while their rolling 7-day spend stays above \$1,000. Set to `168` for weekly cadence, `0` to disable.